

PART 1 OF 5. BEFORE YOU BUILD ANYTHING.

Find the thing worth automating

Most people who want to automate part of their job start in the wrong place. They start with a tool. They read about an API, or somebody shows them an agent demo, and they go looking for a problem that fits it.

Start at the other end. Start with the sentence you type most often.

I am a senior technical program manager at Microsoft. The worst part of that job was chasing people for status, so I automated it. Before any of that worked, I had to pick the right target and get three boring things in place. That is what this document is. No code, nothing to install yet, and it works whether your team lives in Jira, Azure DevOps, HSDes, a Slack channel or a spreadsheet somebody guards.

1 Write down the sentence you type most weeks

For one week, keep a note open and log every time you ask somebody for information you could have looked up. Not decisions. Not opinions. Information.

At the end of the week you will have the same sentence written down eight or nine times in slightly different words. Mine was some version of "is this still on track". Yours might be "did the build pass", "who owns this now", or "has finance approved it yet".

That repeated sentence is your target. It is not the most interesting problem you have. It is the one that costs you the most time.

EXAMPLE

My week looked like this. Monday, four people asked for status. Tuesday, two. Wednesday, six, plus a meeting where I read the same list out loud. That is twelve versions of one question in three days.

WHY THIS MATTERS: automation dies when it starts from the tooling, because you end up building something impressive that saves you twenty minutes a month. Starting from a sentence you actually type keeps it small enough to finish and big enough to be worth it.

2 Put your candidate through three tests

Not every annoying task is worth automating. Three questions, and it has to pass all three.

How often does it happen. Weekly at least. If it is monthly, you will spend more time building than you ever save, and you will not remember how it works when it breaks.

Does it need a judgment. If answering the question requires you to decide whether something matters, stop. A program can tell you an item is late. It cannot tell you whether that is fine. Automate the collecting, never the deciding.

Does the answer already exist somewhere. Somebody has to be typing it into a system already. If the information only lives in people's heads, no tool can reach it and you have a different problem.

EXAMPLE

"Is this on track" passes. It happens daily, the answer is a date and a state that already sit in the tracker, and deciding what to do about a slip stays with me. "Is this feature a good idea" fails on the second test and always will.

WHY THIS MATTERS: the tasks that pass all three are boring, repetitive and invisible. That is exactly why nobody has automated them yet, and why doing it is worth so much.

3 Ask for read access, and ask for it properly

Whatever system holds the answer has a way to read it programmatically. Jira has a REST API. Azure DevOps has one. HSDes has one. Slack has one. Even a spreadsheet on a shared drive can be read by a script.

Read access is a much smaller ask than people think, because you are not changing anything. The mistake is asking vaguely. Ask for a specific thing, say what for, and say what you will not do.

EXAMPLE

Hey, I want to build a small reporting script for our own team. Can I get read-only API access to our project? I only need to pull the owner, the state and the target date. I am not writing anything back, and it stays inside the team.

WHY THIS MATTERS: this is the step people stall on for months, usually because they are waiting to feel ready to ask. It is normally one message and a token, not a project.

4 Confirm you can get three fields

Before you build anything, prove you can pull three things for every item you care about. Who owns it. What state it is in. When it is due.

Owner, state, date. That is it. If you can get those three, everything in the next four parts of this series is possible. If you cannot, find out which one is missing and why, because that is a real finding about how your team works and it is worth surfacing on its own.

EXAMPLE

Half the items in my tracker had no target date at all. That was not a tooling problem, it was that nobody had ever been asked for one. Finding that out was more valuable than the script.

WHY THIS MATTERS: most automation projects die at the data, not at the code. Ten minutes proving the fields exist saves you a weekend of building on sand.

5 Set up a workbench you can actually use

You do not need to be an engineer for this. I am not one. I have a computer engineering degree and I have not written production code in years.

Install a code assistant you can talk to in plain language. I used Claude Code. Install Python if it is not already there. Get your API token from step three and put it somewhere your script can read it that is not the script itself.

Then describe what you want in a sentence, the same way you would to a colleague, and let it write the first version.

EXAMPLE

Here is the API for our tracker. Write me a script that pulls every open item with an owner, a state and a target date, and prints the ones where the date has passed and the state is not closed.

WHY THIS MATTERS: the barrier to this stopped being programming ability about two years ago. The barrier now is knowing precisely what you want, which is why the first four steps come before this one.

None of this is clever. It is a sentence, three tests, one access request and a workbench. The hard parts are all human.

Do the week of logging before part 2. You will not need it to follow along, but you will need it to build your own version, and the version you build for your own job is the only one that matters.

Part 2 is next: turning that sentence into a rule a program can run. That is where most people find out their question was never really a question.